

Name: Arava Raja

Reg No: 192365033

Course Code: CSA1718

PROGRAM – 3

Cript Arithmetic Algorithm

Aim

To develop a Python program that solves a **Crypt Arithmetic Problem** using the **Backtracking (Brute Force)** algorithm by assigning unique digits to letters such that the arithmetic equation is satisfied.

Algorithm

1. Define the letters involved in the crypt arithmetic puzzle (S, E, N, D, M, O, R, Y).
2. Generate all possible unique digit assignments (0–9) for these letters.
3. Assign one unique digit to each letter.
4. Ensure that the leading letters (S and M) are not assigned the digit 0.
5. Convert the words **SEND**, **MORE**, and **MONEY** into their corresponding numerical values.
6. Check whether the equation **SEND + MORE = MONEY** is satisfied.
7. If the equation is true, display the letter-to-digit mapping and the corresponding numerical values.
8. Otherwise, continue checking the remaining digit assignments.
9. Repeat the process until a valid solution is found or all possible combinations have been checked.
10. Stop the program.

Python Program

```
from itertools import permutations
```

```
def solve_crypt():

    letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

    for perm in permutations(range(10), len(letters)):

        d = dict(zip(letters, perm))

        # Leading letters should not be zero

        if d['S'] == 0 or d['M'] == 0:

            continue

        send = d['S']*1000 + d['E']*100 + d['N']*10 + d['D']

        more = d['M']*1000 + d['O']*100 + d['R']*10 + d['E']

        money = d['M']*10000 + d['O']*1000 + d['N']*100 + d['E']*10 + d['Y']

        if send + more == money:

            print("Solution Found\n")

            print("Letter Mapping")

            for k, v in d.items():
```

```
        print(k, "=", v)

    print("\nVerification")

    print("SEND =", send)

    print("MORE =", more)

    print("MONEY =", money)

    return

print("No Solution Exists")

solve_crypt()
```

Expected Output

Solution Found

Letter Mapping

S = 9

E = 5

N = 6

D = 7

M = 1

O = 0

R = 8

Y = 2

Verification

SEND = 9567

MORE = 1085

MONEY = 10652

PROGRAM – 4

Missionaries and Cannibals

Aim

To develop a Python program to solve the **Missionaries and Cannibals Problem** using the **Breadth-First Search (BFS)** algorithm without violating the safety constraints.

Algorithm

1. Represent each state as (**Missionaries Left, Cannibals Left, Boat Position**).
2. Define the initial state as (**3, 3, Left**) and the goal state as (**0, 0, Right**).
3. Create a queue and insert the initial state into it.
4. Mark the initial state as visited.
5. Remove a state from the queue.
6. Generate all possible valid moves of the boat.
7. Check whether the new state is valid (missionaries should never be outnumbered by cannibals on either bank).
8. If the new state is the goal state, display the solution path.
9. Otherwise, add the valid unvisited states to the queue.
10. Repeat until the goal state is reached or the queue becomes empty.

Python Program

```
from collections import deque
```

```
# Check if a state is valid
```

```
def is_valid(m_left, c_left):
```

```
m_right = 3 - m_left
```

```
c_right = 3 - c_left
```

```
if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
```

```
    return False
```

```
if (m_left > 0 and m_left < c_left):
```

```
    return False
```

```
if (m_right > 0 and m_right < c_right):
```

```
    return False
```

```
return True
```

```
# BFS Algorithm
```

```
def missionaries_cannibals():
```

```
    start = (3, 3, 'L')
```

```
    goal = (0, 0, 'R')
```

```
    queue = deque([(start, [])])
```

```
    visited = set()
```

```
    while queue:
```

```
        (m_left, c_left, boat), path = queue.popleft()
```

```
        if (m_left, c_left, boat) in visited:
```

```
            continue
```

```
        visited.add((m_left, c_left, boat))
```

```
        path = path + [(m_left, c_left, boat)]
```

```

        if (m_left, c_left, boat) == goal:

            print("Solution Found!\n")

            for state in path:

                print(state)

            return

moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]

for m, c in moves:

    if boat == 'L':

        new_state = (m_left-m, c_left-c, 'R')

    else:

        new_state = (m_left+m, c_left+c, 'L')

    if is_valid(new_state[0], new_state[1]):

        queue.append((new_state, path))

print("No Solution Exists")

# Run Program

missionaries_cannibals()

```

Expected Output

Solution Found!

```

(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')

```

(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')